Assignment 1

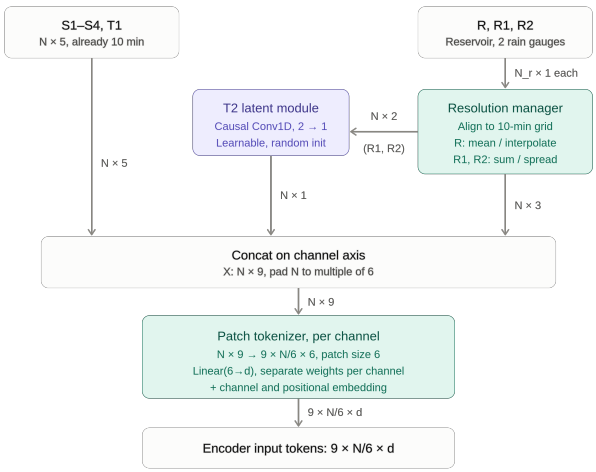
Zainab-Binte-Iftikhar (25280064) [1]

Abstract

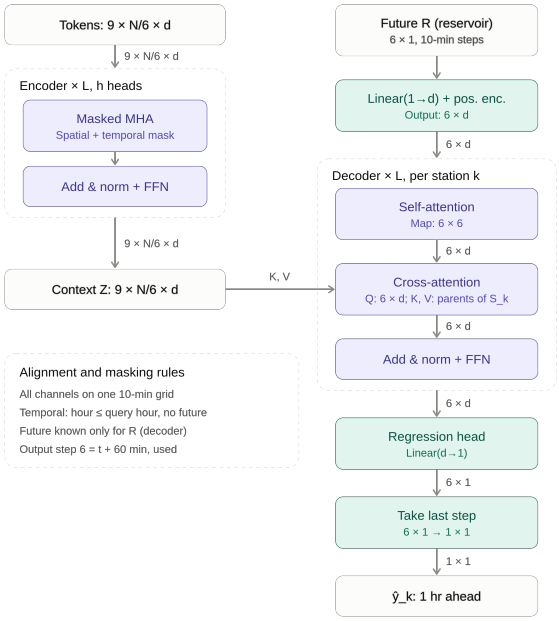
Below are solutions to Assignment 2 of AI651 – Deep Learning for Space, Time and Graphs. All questions are answered with visual proofs like graphs or tables where necessary.

1. Question 1 - River Forecasting Proposal

1.1. Architecture diagram



. Input processing and tokenizing of different streams of data



. Encoder-Decoder Model

(***Note:*** *The above diagrams were made with Claude but the architecture presented is based on my own understanding and design. One exception is handling of T2 i.e. unmonitored tributory which was brainstormed with Claude)*

1.2. Assumptions

A1. S1 readings are influenced by Reservoir and R1 (from given diagram)

A2. S2 readings are influenced by Reservoir, R1, S1, T1 (from given diagram)

A3. S3 readings are influenced by Reservoir, R1, R2, S1, S2, T2 (unmonitored tributory), T1 (from given diagram)

A4. S4 readings are influenced by Reservoir, R1, R2, S1, S2, S3, T2 (unmonitored tributory), T1 (from given diagram)

A5. Unmonitored tributory is influenced by rain readings R1 and R2.

[1]MS AI, Lahore University of Management Sciences, Lahore, Pakistan. Correspondence to: Zainab-Binte-Iftikhar <25280064@lums.edu.pk>.

A6. Timestamps are available for data of all sensors i.e. S1-S4, R1, R2, Reservoir and T1

1.3. Forecasting task and data specification

Issuance time would be the 't' for which we need a prediction of what would happen at t+60 (an hour ahead).

History/Lookback window taken for this problem is last hour. That will contain 6 tokens. Each of these 6 tokens is produced from a patch of size 6 from original data so each token contains one hour of info.

The horizon here is one-hour ahead of time t (given in the problem statement in the assignment). For the above mentioned model, there will be 6 predictions (each representing values of S1-S4 per 10 mins). We will pick the final output as our prediction. The output will be a 1x4 vector with elements containing water level for S1-S4.

Tensor	**Shape**	**Axis meaning**	**Time coverage**	**Available at issuance (t)**
Inputs				
S_1–S_4, T_1 (raw)	$B \times N \times 5$	batch; time step (10 min); channel	$t-(N-1)\Delta, \ldots, t$	Yes (timestamps $\le t$)
R, R_1, R_2 (raw)	3 series, each $B \times N_r \times 1$	batch; time step at the sensor's own rate; channel	$t-(N-1)\Delta, \ldots, t$	Yes (timestamps $\le t$)
R, R_1, R_2 (aligned)	$B \times N \times 3$	batch; time step (10 min); channel (R, R1, R2)	$t-(N-1)\Delta, \ldots, t$	Yes, history only
Future R (decoder input)	$B \times 6 \times 1$	batch; future step (10 min); channel	$t+\Delta, \ldots, t+6\Delta$	Assumed known at issuance. Future S, T_1, R_1, R_2 are not
Intermediate tensors				
T_2 module input (R_1, R_2)	$B \times N \times 2$	batch; time step (10 min); channel (R1, R2)	$t-(N-1)\Delta, \ldots, t$	Yes, history only
T_2 (learned latent)	$B \times N \times 1$	batch; time step (10 min); channel (T_2)	$t-(N-1)\Delta, \ldots, t$	Never observed; computed from rain history
Concatenated X	$B \times N \times 9$	batch; time step; channel (S_1–S_4, T_1, T_2, R, R_1, R_2)	$t-(N-1)\Delta, \ldots, t$	Yes, history only
Encoder tokens / context Z	$B \times N/6 \times d$	batch; token (6 steps = 1 hour); feature	$N/6$ hourly patches ending at t	Yes, history only
Decoder tokens	$B \times 6 \times d$	batch; future step (10 min); feature	$t+\Delta, \ldots, t+6\Delta$	Known future R plus encoder context
Outputs				
$\hat{y}_{1:6}$ (head output)	$B \times 6 \times 1$	batch; future step (10 min); target	$t+\Delta, \ldots, t+6\Delta$	No (to be predicted)
$\hat{y}$ (final)	$B \times 1$	batch; target	$t+60$ min	No (to be predicted)

1.3.1. Alignment Decisions

The readings for S1-S4 and T1 are available in 10 min time steps. The remaining readings for Reservoir, R1, and R2 rain gauges are not given in this resolution, so we will resample them to be in 10 min reslution. In case of finer resolution available, we will mean over 10 mins and in case of coarser resolution, we will impute data. For patch tokenization, we pad so that output dimension is maintained. For prediction of t+60 at t, we will have all information available at $time \le t$ as well as future info about Reservoir. No future info about any other sensor will be available. As far as the unmonitored tributory is concerned, its values will be predicted based on rain gauge readings through a 1D convolution model.

1.4. Model computations and design justification

1.4.1. Input Block & Tokenzier

Model uses a pre processing block to bring all inputs to the same resolution i.e. 10mins. These modified inputs are then sent to tokenizer to produce token based on patching. I have chosen a patch size of 6. Patching is applied to each cahnnel separtely. The intuition is that each output token will be able to express the information over an hour for a particular channel/sensor. To handle the unmonitored tributory and its influence on S3 and S4, i have added a causal 1D convolution block that uses readings from R1 and R2 to produce the level of T2.

Let $\bar{X} \in \mathbb{R}^{N \times 9}$ be the standardized input, with N a multiple of $P = 6$ and $n_p = N/P$. The patch of channel c at hour ℓ is

$$\mathbf{s}_{c,\ell} = \left[\bar{X}_{(\ell-1)P+1,\,c},\ \ldots,\ \bar{X}_{\ell P,\,c}\right]^\top \in \mathbb{R}^P, \qquad c \in \mathcal{C},\ \ell = 1, \ldots, n_p. \tag{1}$$

Patch embedding for one patch is

$$\mathbf{x}_{c,\ell} = W_c\, \mathbf{s}_{c,\ell} + \mathbf{b}_c + \mathbf{e}_c + \mathbf{p}_\ell \ \in \mathbb{R}^d, \tag{2}$$

where

- $W_c \in \mathbb{R}^{d \times P}$ and $\mathbf{b}_c \in \mathbb{R}^d$ are the weights and bias of the linear patch map for channel c (separate for each channel);
- $\mathbf{e}_c \in \mathbb{R}^d$ is a learned channel embedding, which tells the encoder which sensor a token comes from (it is also what the spatial mask refers to);
- $\mathbf{p}_\ell \in \mathbb{R}^d$ is a learned positional embedding for hour ℓ, shared across channels.

Stacking all tokens gives

$$X^{\text{tok}} = \left[\mathbf{x}_{c,\ell}\right]_{c \in \mathcal{C},\ \ell = 1..n_p} \in \mathbb{R}^{9 \times n_p \times d}. \tag{3}$$

1.4.2. Encoder Block

Encoder uses 2 types of masking: Spatial and Temporal. The spatial mask allows us to encode the assumptions A1-A4. This way during attention calculations, an S1 token will only attend to what its influenced by i.e. Reservoir and

R1 token and so on. For the temporal mask, we only want to give attention to readings before issuance time *t* for all sensors. In addition to masks, we also use multi headed attention (MHA) and each head uses the same spatial and temporal masks.

Token indexing. Order the channels as $(S_1, S_2, S_3, S_4, T_1, T_2, R, R_1, R_2) \mapsto c = 1, \dots, 9$. Token (c, ℓ) is channel c at hour $\ell = 1, \dots, n_p$, and the tokens are flattened channel-major to $N_{\text{tok}} = 9n_p$ rows.

Mask 1: spatial mask. First define the channel-level matrix $A \in \{0,1\}^{9\times 9}$ from the dependency assumptions:

$$A_{c,c'} = \mathbb{1}\big[\, c' \in \mathrm{Pa}(c) \cup \{c\} \,\big], \tag{4}$$

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{matrix} S_1 \\ S_2 \\ S_3 \\ S_4 \\ T_1 \\ T_2 \\ R \\ R_1 \\ R_2 \end{matrix} \tag{5}$$

Row c lists the channels whose tokens may be read by the tokens of channel c. The spatial mask on the full set of tokens is

$$M^S \in \{0,1\}^{N_{\text{tok}}\times N_{\text{tok}}}, \qquad M^S_{(c,\ell),(c',\ell')} = A_{c,c'} \quad \text{for all } \ell, \ell' \tag{6}$$

That is, each entry $A_{c,c'}$ is repeated over the whole $n_p \times n_p$ block of tokens that belong to channel c (queries) and channel c' (keys).

Mask 2: temporal mask. Temporal mask will be an hour-level matrix $T \in \{0,1\}^{n_p\times n_p}$:

$$T_{\ell,\ell'} = \mathbb{1}\big[\, \ell' \le \ell \,\big], \tag{7}$$

which is lower triangular. For $n_p = 4$:

$$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}. \tag{8}$$

The temporal mask on the full set of tokens is

$$M^T \in \{0,1\}^{N_{\text{tok}}\times N_{\text{tok}}}, \qquad M^T_{(c,\ell),(c',\ell')} = T_{\ell,\ell'} \quad \text{for all } c, c' \tag{9}$$

Additive forms.

$$B^S_{ij} = \begin{cases} 0, & M^S_{ij} = 1, \\ -\infty, & M^S_{ij} = 0, \end{cases} \qquad B^T_{ij} = \begin{cases} 0, & M^T_{ij} = 1, \\ -\infty, & M^T_{ij} = 0. \end{cases} \tag{10}$$

Applying the two masks. For each head a of every encoder layer, with $Q_a, K_a, V_a \in \mathbb{R}^{N_{\text{tok}}\times d_h}$, the raw scores are $S_a = Q_a K_a^\top/\sqrt{d_h}$ and the masks are applied one after the other:

$$S_a^{(1)} = S_a + B^S \qquad \text{(spatial mask)}, \tag{11}$$

$$S_a^{(2)} = S_a^{(1)} + B^T \qquad \text{(temporal mask)}, \tag{12}$$

$$\mathrm{head}_a = \mathrm{softmax}\big(S_a^{(2)}\big)\, V_a. \tag{13}$$

Entry (i,j) is finite only if both $M^S_{ij} = 1$ and $M^T_{ij} = 1$, so

$$\alpha_{ij} = \frac{M^S_{ij}\, M^T_{ij}\, e^{s_{ij}}}{\sum_{j'} M^S_{ij'}\, M^T_{ij'}\, e^{s_{ij'}}}, \tag{14}$$

where s_{ij} is the raw score. The same B^S and B^T are used for every head, every layer, and every sample in the batch.

1.4.3. Decoder Block

In decoder, we have simple self attention as well as cross attention. Within the decoder block, we want to ensure that the data provided is masked in a way such that all sensor info except for Reservoir is causal.

Decoder self-attention (temporal mask only). The keys and queries are the six future reservoir tokens, $j, j' = 1, \dots, 6$, which all belong to channel R. The spatial mask is trivial, $M^{S,\text{self}}_{j,j'} = A_{R,R} = 1$. The temporal mask encodes that future values exist only for R and are all known at issuance:

$$M^{T,f}_{j,j'} = 1 \quad \text{(bidirectional)}, \qquad \text{or} \qquad M^{T,f}_{j,j'} = \mathbb{1}\big[\, j' \le j \,\big] \quad \text{(causal)}, \tag{15}$$

$$\mathrm{head}_a = \mathrm{softmax}\Big(\frac{Q_a K_a^\top}{\sqrt{d_h}} + \log M^{T,f}\Big) V_a, \qquad Q_a, K_a, V_a \in \mathbb{R}^{6\times d_h}. \tag{16}$$

Only R has info at future steps. For all other sensors, we dont have info for future so their entries at future steps will be 0 when provided in input.

Application in the decoder for target station S_k. Let $\mathbf{a}_k$ be the S_k row of A, so that a key of channel c' is visible to the queries of S_k only if $a_{k,c'} = 1$. The spatial mask on the decoder keys is

$$M^{S,\text{dec},k}_{j,(c',\cdot)} = a_{k,c'}, \tag{17}$$

and it is applied together with the temporal mask:

$$S^{(1)} = \frac{\tilde{Q} K^\top}{\sqrt{d_h}} + \log M^{S,\text{dec},k} \qquad \text{(spatial)}, \tag{18}$$

$$S^{(2)} = S^{(1)} + \log M^{T,\text{dec}} \qquad \text{(temporal, including future data)}, \tag{19}$$

$$\mathrm{head}_a = \mathrm{softmax}\big(S^{(2)}\big) V. \tag{20}$$

Since $a_{k,R} = 1$ for every station (R is a parent of all of them), the future reservoir keys are visible to the queries of every station.

Decoder works in an autoregressive function like above to produce outputs at 6 steps after *t* with each step equivalent to 10mins. These predictions are all passed into a regression head to produce output at t+6 (an hour after t).

1.4.4. Final Regression Head

The decoder embeddings are at the end fed into a regression head that produces a 4 dimensional vector containing water levels from S1-S4. As outputs are produced in an autoregressive fashion so we pick the output at last step for our final prediction.

The decoder output for target station S_k is $D^{(k)} \in \mathbb{R}^{6\times d}$ with rows $\mathbf{d}_j^{(k)}$ for the future steps $t + j\Delta$, $j = 1, \ldots, 6$. A linear head, with separate parameters for each station, maps each row to a standard prediction:

$$\hat{y}_{k,j} = \mathbf{w}_{o,k}^{\top}\, \mathbf{d}_j^{(k)} + b_{o,k} \tag{21}$$

$$\mathbf{w}_{o,k} \in \mathbb{R}^d,\ b_{o,k} \in \mathbb{R}, \quad j = 1, \ldots, 6. \tag{22}$$

The final one-hour-ahead forecast is the sixth step:

$$\hat{y}_k = \hat{y}_{k,6} \tag{23}$$

1.5. Operation and exceptional conditions

1.5.1. Missing Observations

Missing observations within the provided sensor data will be handled through mean imputation. The missing data of unmonitored tributory has been handled through a learnable convolution block in the input processing block.

1.5.2. Delayed Observations

For delayed observations, i believe model will be able to learn the delayed impacts through attention across time. No explicit arrangement has been made to encode this information within the proposed architecture.

1.5.3. Announced Future Releases

For announced future releases of the reservoir, decoder architecture has been designed in a way that this can be handled. The information for next 6 timesteps after the issuance time *t* for reservoir are provided as an input to the decoder as well as historic information for reservoir as well as other sensos is also provieded. This is achieved through a temportal mask in the self attention block of decoder where only reservoir readings are allowed to be bidirectional while all other readings are causal.

1.5.4. Differences between scheduled and actual releases

There is nothing in the proposed architecture that explicitly handles this scenario. Perhaps this will be part of noise for the model.

1.5.5. Justification for Running every 10 mins

Consider the following to be variable definitions

Symbol	Definition
B	Batch size
N	Number of 10-minute history steps (a multiple of 6)
Δ	Time interval step ($\Delta = 10$ minutes)
t	Forecast issuance time
d	Model hidden width (embedding dimension)
L	Number of layers in the encoder and decoder
C	Number of input channels ($9inourcase$)
P	Patch size for tokenization ($P = 6$, corresponding to 1 hour)
S_e	Total number of encoder input tokens ($S_e = C \times \frac{N}{P}$)
S_d	Number of future decoder steps ($S_d = 6$, corresponding to 1 hour)

. Summary of mathematical notations and dimensions.

Now using the above notations, we can represent the complexity of the 3 major blocks of our architecture:

- **Block 1 (Preprocessing & Tokenization):** $\mathcal{O}(N \cdot d)$
- **Block 2 (Transformer Encoder):** $\mathcal{O}\left(L \cdot S_e^2 \cdot d\right) = \mathcal{O}\left(L \cdot \left(\frac{C\cdot N}{P}\right)^2 \cdot d\right)$
- **Block 3 (Decoder & Regression Head):** $\mathcal{O}\left(L \cdot S_d \cdot S_e \cdot d + S_d \cdot d\right)$

So, the overall complexity becomes:

Overall Time Complexity = Block 1 + Block 2 + Block 3
$= \mathcal{O}\left(N \cdot d + L \cdot \left(\frac{C\cdot N}{P}\right)^2 \cdot d + L \cdot S_d \cdot S_e \cdot d\right)$

1.6. Uncertainty, limitations, and additional information

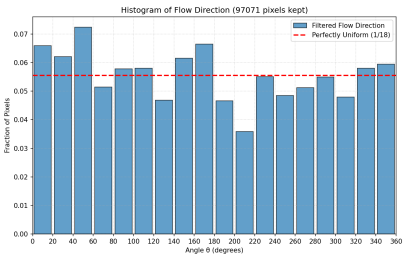
A limitation of the proposed model (as mentioned above in section 1.5) is that it cannot explicitly handle changing travel times and delayed observations. It also cannot deal if there is a difference in when reservoir was supposed to be opened and when it actually was. Problems like these will be difficult to be handled by current architecture and to be honest im not sure how to deal with them. Perhaps for reservoir, we could request historical data of mismatch between scheduled and actual operations to infer a rule from them. I believe the system still remains usable without this info though.

2. Question 2 - Motion or Memory?

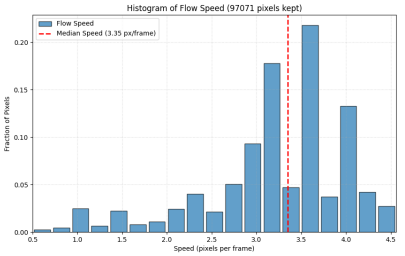
Table 1

	Check	Expected	Yours
(a)	$\max \lvert M(M(x)) - x \rvert$	0	
(b)	$\max \lvert R^4(x) - x \rvert$	0	
(c)	$\max \lvert MRM(x) - R^{-1}(x) \rvert$	0	
(d)	bright pixel of $M(p)$ (frame, row, column)	$(7, 10, 43)$	
(e)	bright pixel of $R(p)$ (frame, row, column)	$(7, 43, 10)$	
(f)	$1\times$ input / target frames	1–5 / 6–10	
(g)	$2\times$ input / target frames	$1, 3, 5, 7, 9$ / $11, 13, 15, 17, 19$	
(h)	$3\times$ input / target frames	$1, 4, 7, 10, 13$ / $16, 19$	

2.1. Figure 1



. Histogram of flow direction



. Hsitogram of flow Speed

Q2.1 Based on Figure 1(a), would mirrored and rotated trajectories already be familiar to a model trained on this data? (Hint: how would the direction histogram change if every trainingvideo were mirrored? If every video were rotated by 90◦ ?) From Figure 1(b), what is atypical speed, and what do 2× and 3× correspond to in pixels per frame? Optical flow is unreliable where digits overlap; could that bias your histograms enough to change your answer?

Answer: If all training videos were mirrored then the histogram would flip horizontally and peaks will be chnaged. Similarly, if training data was rotated by 90 degrees then movement angles of the moving digits would change. e.g. if a digit was moving right and was flipped by 90 degrees, it would start moving down now which would completely change the angle distribution present in 1(a). So for these new distributions, model would be unfamiliar. If original distribution was uniform, then model would have been fine because rotation and mirroring would keep distribution same.

From fig 1(b), typical speed is 3.35 px/frame. With 2x or 3x, the speed would increase because number of frames in a sample would reduce and the change in position of pixels in each frame would be a lot.

Since there are QUITE A LOT (48/50) of overlapping sequences so yes it would change our histographs in this case.

Q2.2 For each transformation, does it change the motion statistics, the digit appearance, or both? Why does this make the three of them useful as a controlled experiment?

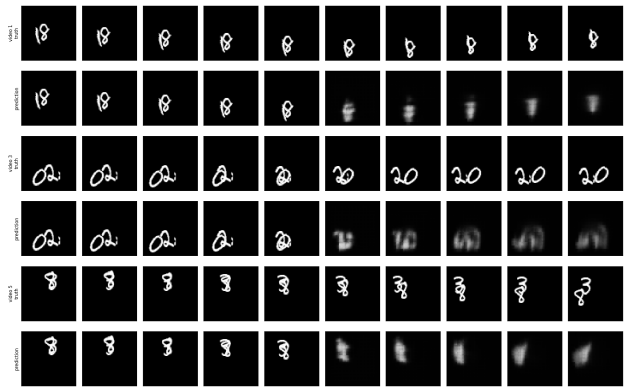
Answer: For both mirroring and rotationm, the digit appearance would change as well as motion stats.For speed changes, only motion stats change, not the appearance. Since the behavior of moving digits is a bit different after these transformations, they could prove useful to test the generalizability of our model. If model fails on transformed images, we have a guide as to where the model experiences trouble.

2.2. Table 2

. Performance Metrics and Parameters Comparison

Metric	Model A Total	Model A (seed0, seed1)	Blank	Copy-last
MSE1	0.0163	(0.0164, 0.0162)	0.0426	0.0488
MSE2	0.02	(0.0202, 0.0198)	0.0426	0.0605
MSE3	0.0226	(0.0228, 0.0223)	0.0426	0.0665
MSE4	0.0245	(0.0248, 0.0243)	0.0426	0.0706
MSE5	0.0263	(0.0266, 0.0260)	0.0426	0.0730
E5	0.0219	(0.0222, 0.0217)	0.0426	0.0639
Params		480784	0	0

2.3. Figure 2



. input, prediction and ground truth for three test videos

Q2.3: Estimate model A's reach in original pixels per time step, using the conventions above. Walk through your reasoning from the patch size, kernel sizes, number of layers and the recurrence.There is no formula to look up for this.

Answer: starting out with folded grid where our 1x64x64 grid patch was converted into 16x16x16. So now one pixel corresponds to 16 original pixels that have been shifted to be channels. In each direction, the reach/distance in this case would be 4 covered now by 1 pixel in the folded grid. So after on conv layer, the distance observed by model is 1 in each direction so reach becomes:

reach1 = 4x1 = 4px

Then after the next conv layer, the grid dimensions remain same because of padding and 1 stride. But the area observed by the model increases 1 step further in each direction and becomes 2. So the reach/distance now becomes:

reach2 = 4x2 = 8px

Afterwards, we ahave 1x1 convolutions which dont move info so we can ignore those. So Model A's reach becomes 8 pixels (in each direction).

Q2.5: Before you run any experiment in Testing the Baseline, write down your predictions: will model A pass the mirror test? The rotation test? At what speed should it break down, given your reach estimate and Figure 1? Include this answer in your report exactly as you first wrote it, even if it turns out to be wrong.

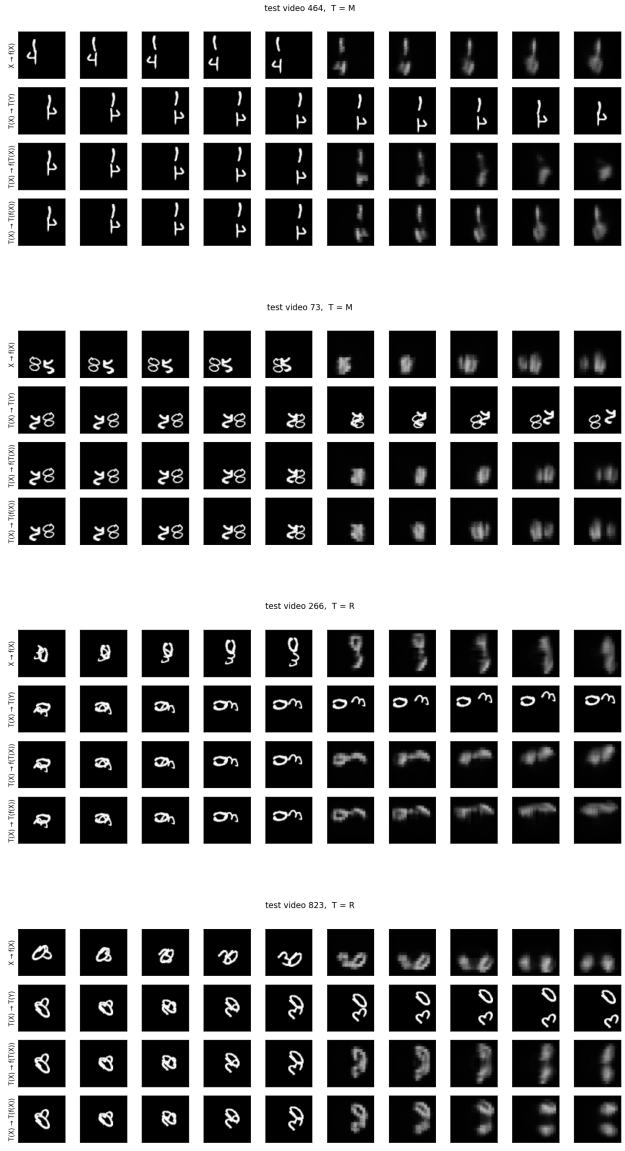
Answer: *(can be verified from GitHub file **Ques2.ipynb**)* I think Model A would fail both rotation and mirror test because in both these cases, the appearance of the digits will change. Since model A was only trained on digits aligned in one direction, flipping or rotating will completely change the orientation and look and model will fail. Even more so because it already has a hard time predicting the detailed images of digits (blurry predictions). For speed, i think it will fail at 3x. Relating it to reach, for 1x, speed is 3.3pixels (from Figure 1). For 2x, it will become 6.6 (medianx2). These are both values well within our model's reach i.e. 8pixels. For 3x, however, the speed will become 9.9 pixels which exceeds our model's reach and so our model wont be able to follow the digit motion very well.

2.4. Table 3

. Performance Metrics Comparison

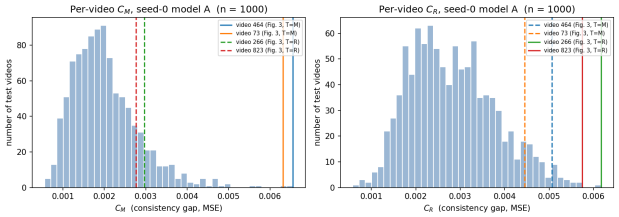
Metric	**Model A**			**Blank**	**Copy-last**
	Mean	**s0**	**s1**		
E5	0.0219	0.0222	0.0217	0.0426	0.0639
C_M	0.00206	0.00172	0.00239	0	0
E_M	0.0225	0.0227	0.0222	0.0426	0.0639
C_R	0.00310	0.00301	0.00319	0	0
E_R	0.0236	0.0239	0.0233	0.0426	0.0639

2.5. Figure 3



. the two test videos with the largest per-video CM (with T = M) and the two with the largest per-video CR (with T = R), for the seed-0 model A

2.6. Figure 4



. histograms of the per-video CM and CR of the seed-0 model A over the test set, with the videos from Figure 3 marked.

Q2.6: Blank and Copy-last are exactly consistent under mirror and rotation. What does this say about reading CT on its own, and why does ET always have to be reported next to it?

Answer: Just because prediction are consistent (whether you predict and then apply transformation or transform and predict) does not mean that the predictions are correct. C_T just shows that the prediction function behaves the same whether transformation is applied at start or end, it does not show the correctness of prediction function and whether it is good or bad (evident through 0 C_T for both cases and non zero E_T). This is why we also need E_T so we can learn about the correctness of our predictions as well.

Q2.7: Pick one video in Figure 3 where rows 3 and 4 disagree and describe precisely how the model's account of the motion changes. For example, a digit goes the wrong way, gets lost,bounces too early or not at all, or blurs into something unrecognisable.

If we consider the test video 823 with *Rotation* as the transformation, we can see that from the get go the digits are blurry in both rows 3 and 4 but the blurriness and digit mutation in row 3 (i.e. the row with transformation is applied *before* the prediction function) is much more significant than row 4. The digit orientation is also wrong in row 3. Its almost at a 90 degree angle whereas the true result is titled. Row 3 has also kept very little distance between the 2 digits whereas in the true result, there is a separation between them.

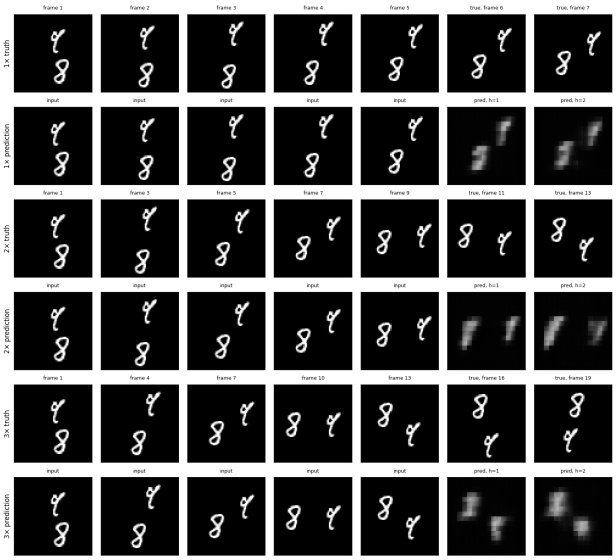
Q2.8: Is rotation harder than mirroring for model A? Using your answers to Q2.1 and Q2.2, argue whether the difference comes from the motion or from the digits' appearance. From Table 3, we can see that error in case of rotation is a little bit more than mirroring. So, rotation is a bit more difficult for our model than mirroring. The difference between them is minor. I believe the difference is because of digit appearance because motion is actually captured decently by the model (observed in Figure 3) but appearance keeps getting distorted as we move through time.

2.7. Table 4

. Performance Comparison Across Speed Configurations

	E2 (1x)	E2 (2x)	E2 (3x)	E5 (1x)	E5 (2x)
Model A (seed 0, seed 1)	0.0176 (0.0203, 0.0149)	0.0449 (0.0425, 0.0473)	0.0513 (0.0493, 0.0532)	0.0210 (0.0235, 0.0185)	0.0487 (0.0470, 0.0504)
Copy-last	0.0545	0.0657	0.0701	0.0640	0.0717

2.8. Figure 5



. Five input frames and the first two predicted and true future frames at 1×, 2× and 3× (seed-0 model A) for one test video

Q2.9: At what speed do model A's predictions fall apart, and does this agree with the reach you estimated in Q2.3? If not, what did your estimate miss? Describe what the failure in Figure 5 looks like (blur, a digit that stalls, a duplicated digit, a digit that vanishes) and what each of these suggests the model is doing.

Answer: As far as movement is concerned, model A's predictions start falling apart at 2x and get even worse at 3x. It doesnt agree with my estimate in Q2.3 (reach of 8px). This is because even though the model *could* in theory look that far, it doesnt mean it actually *does* look that far. There is no guarantee in the architecture.

There are multiple failures in Figure 5: the digits are blurred and their movement and position is wrong as comapred to true values. This means the model has a hard time figuring out the exact details of the digit appearances. Though the movement is fine at 1x, 2x onwards, that fails as well with model just repeating the previous frame's positions with very minor adjustments.

Q2.10: From these results alone, can you tell whether the speed failure happens because fast motion never appeared in training, or because it is beyond the model's reach? Propose an experiment that would tell the two apart.

Answer: We can't answer for sure unless we compare both in isolation but an ides is that in training data, the median speed our model saw was around 3px/frame with max speed at 4.5px/frame. For 2x and 3x speed, these metrics would double and triple and go beyond the values that our model saw during training. So I think its because model did not train on these speed variations.

One experiment to test this could be to train the model on some sped up versions of the data so e.g. 1/3 of data at 1x, 1/3 at 2x and remaining at 3x. This would help us see whether the issue is model's reach or training data.

Q2.11: Before training B–D, predict in at most three sentences which model will fix which of the failures you found in Testing the Baseline. As with Q2.5, include this unedited in your report.

Answer: *(can be verified from GitHub file **Ques2.ipynb**)* Model B will fix performance on rotated and mirrored images and hopefully reduce blur. Model C and D will fix performance on 2x and 3x speeds regarding digit positions.

2.9. Table 5

. Comprehensive Performance Comparison across Models

Metric	A	B	C	D_dilation	DB
E(5) 1x	0.0218 (0.0221, 0.0214)	0.0217 (0.0218, 0.0217)	0.0281 (0.0287, 0.0275)	0.0273 (0.0274, 0.0273)	0.0278 (0.0277, 0.028)
C_M	0.002 (0.00167, 0.00232)	0.00131 (0.00123, 0.00138)	0.00101 (7.40e-04, 0.00129)	5.95e-04 (6.34e-04, 5.55e-04)	3.42e-04 (4.23e-04, 2.61e-04)
E_M	0.0224 (0.0227, 0.0221)	0.0218 (0.0219, 0.0218)	0.0282 (0.0288, 0.0276)	0.0274 (0.0275, 0.0274)	0.0278 (0.0277, 0.028)
C_R	0.00309 (0.00303, 0.00314)	0.00164 (0.00156, 0.00172)	0.00217 (0.00173, 0.00261)	0.00174 (0.00152, 0.00197)	3.33e-04 (3.91e-04, 2.74e-04)
E_R	0.0234 (0.0238, 0.0231)	0.0217 (0.0217, 0.0216)	0.0291 (0.0296, 0.0286)	0.0283 (0.0282, 0.0284)	0.0279 (0.0277, 0.0281)
E(2) 1x	0.0181 (0.0185, 0.0178)	0.018 (0.0179, 0.0181)	0.0239 (0.0242, 0.0236)	0.0244 (0.0243, 0.0245)	0.0246 (0.0243, 0.025)
E(2) 2x	0.045 (0.0442, 0.0458)	0.0446 (0.0438, 0.0454)	0.0297 (0.0298, 0.0295)	0.0293 (0.0293, 0.0293)	0.0295 (0.0294, 0.0297)
E(2) 3x	0.0519 (0.0508, 0.053)	0.05 (0.0477, 0.0523)	0.034 (0.0341, 0.0339)	0.0327 (0.0326, 0.0328)	0.0328 (0.0327, 0.0329)
Params	480784	480784	480784	480784	480784
Train s/epoch	n/a	n/a	n/a	n/a	20.2 (20.2, 20.2)
Inference s	0.392 (0.388, 0.397)	0.393 (0.392, 0.395)	0.394 (0.388, 0.399)	0.548 (0.546, 0.549)	0.528 (0.526, 0.531)

Q2.12: Describe model D and estimate its reach the same way you did in Q2.3, showing that it is strictly larger than model A's.

Answer: Model D chosen here is the one where we added dilation of 2 in the convolutions while keeping everything identical to the baseline model A. In this case, we applied a dilation of 2 only to the second layer (intuition is that first layer will then be able to capture digit detail and second can help increase reach).

The input to the model remains same as A with 1x64x64 grid patch converted into 16x16x16. one pixel corresponds to 16 original pixels that have been shifted to be channels. Since patch size is 4, the distance covered by 1px movement in first layer will then corresponse to 4 pixels in the original unfolded gros. So after one conv layer, the reach of model D is same as A i.e.

$\texttt{reach1_D} = 4 \times 1 = 4\,\text{px}$

But now in 2nd layer, we have a dilation convolution so each one pixel movement corresponds to 8 original pixels

covered. So the reach after layer 2 can then be obtained by adding the 2:

$\texttt{reach2_D} = 4 + 8 = 12\,\text{px}$

so total reach of Model D with dilated conv is 12px which is greater than model A's reach which was 8px.

Q2.13: Label each failure from Testing the Baseline as appearance, unfamiliar motion, or architectural reach, citing the comparison in Table 5 that supports each label. In particular, if model C still does badly at 3× while model D does well, what does that tell you, and why can't more training data carry information further than the architecture allows?

Answer:

- Failure on Mirroring is Appearance Failure because using different orientations in reduces both consistency and transformation error. When we dont provide this data to model C and A, their errors are visibly higher.

- Failure on Rotation is also Appearance failure becasue it is also significantly helped by model B that was fed the different orientations data.

- Failure in faster motiion can be referred to as due to unfamiliar motion and reach failures because in Model C (where we augmented with various speed data) and Model D (where we increased reach with dilation), the errors for 2x and 3x speeds are much lesser than A and B.

Model C doesn't really do too badly. Its performance is comparable with our chosen model D. This shows that even thought D has a much higher reach than C, the significant reach did not provide a relative significant gain in performance.

Q2.14: Did any of these changes cost accuracy on the original 1× clips? If so, was it worth it?

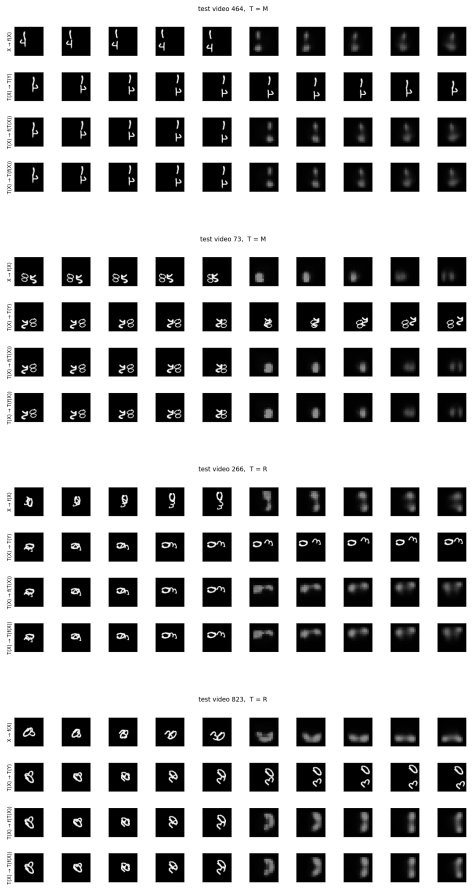
Answer: Yes model C and D showed higher errors on 1x as compared to models A and B. But I believe this was worth it because it allowed model to perform better with videos at a higher speed i.e. at 2x and 3x.

2.10. Table 6

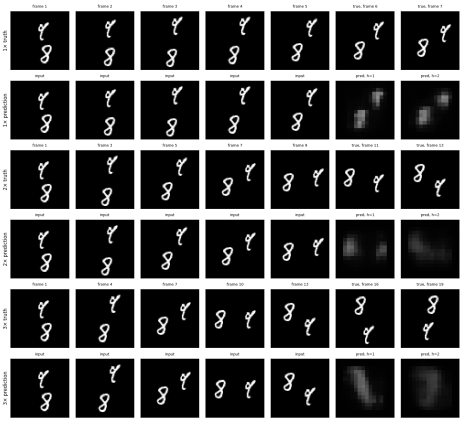
. Performance Comparison for Symmetric Models

Metric	Sym-B	Sym-D (final)
E(5) 1x	0.0211 (0.0212, 0.021)	0.0273 (0.0273, 0.0273)
C_M	8.80e-17 (7.58e-17, 1.00e-16)	5.97e-17 (5.79e-17, 6.14e-17)
E_M	0.0211 (0.0212, 0.021)	0.0273 (0.0273, 0.0273)
C_R	7.12e-17 (6.10e-17, 8.13e-17)	4.74e-17 (4.66e-17, 4.83e-17)
E_R	0.0211 (0.0212, 0.021)	0.0273 (0.0273, 0.0273)
E(2) 1x	0.0176 (0.0175, 0.0177)	0.0244 (0.0243, 0.0245)
E(2) 2x	0.0438 (0.0429, 0.0446)	0.0293 (0.0293, 0.0294)
E(2) 3x	0.0489 (0.0467, 0.0511)	0.0326 (0.0325, 0.0328)
Params	480784	480784
Train s/epoch	n/a	n/a
Inference s	3.36 (3.32, 3.4)	5.04 (4.91, 5.17)

2.11. Figure 6



. Figure 6 (a) - Figure 3 repeated with your final model on exactly the same videos



. Figure 6 (b) - Figure 5 repeated with final model on exactly the same video

Q2.15: Compare symmetry that is learned (model B) with symmetry that is built in (the symmetrised models) in terms of consistency, accuracy on original and transformed clips, and inference cost. Which one is exact, and why? Averaging eight predictions can also blur them; when would you expect built-in symmetry to hurt accuracy, and did it here?

Answer: In terms of consistency, the symmetrised model overtake the leaned model B with almost 0 consistency errors. In terms of accuracy, the symmetrized models however produce blurrier results as compared to the original models with learned symmetry. This is true for both original and transformed clips. Inference cost of symmetrized models is also much higher as compared to learned model B.

Symmetrized models are exact as compared to learned model because in learned model, there is no architectural constraint that forces the model to learn symmtry as opposed to symmetrized model. Averaging eight predictions definitely blurred results here as can be seen when you compare predictions for symmetrized models vs original model B. Built in symmetry hurts overall accuracy here but only a little bit because it didnt learn symmetry as much as symmetrized models were forced to learn.

Q2.16: Go back to the video you discussed in Q2.7. Does the final model now get it right (Figure 6), and which change fixed which part of the failure?

Answer: Yes the final model works as far as the consistency between row 3 and 4 of video 823 (T=R) is concerned. I believe feeding data with different speeds (1x, 2x, 3x) helped model learn digit motion better and ended up orienting them in a way that agrees a lot more than model A.

Q2.17: Go back to the video you discussed in Q2.7. Does the final model now get it right (Figure 6), and which change fixed which part of the failure?

Answer: Yes i believe my final model (i.e. Wrapped Model D with dilation) has learned motion. This can be observed through comparison of Figure 3 with Figure 6(b). The orientations are now much more correct than before. Consistency has been improved significantly as compared to other models in case of both Mirroring and Rotation. The model also performs much better on 2x and 3x videos than the baseline model (even if it is at a slight loss in 1x accuracy).

The only thing that is still a concern is the carry forward error that blurs digits beyond recognition.

A. Code Repository and Implementation Details

You can access the repository on GitHub at:
Code_for_assg
This repo contains the notebooks for both Task 1 and 2.

B. AI Usage Statement

In completing this assignment, I used an AI assistant (Claude, by Anthropic) in the following ways:

1. **Understanding the assignment:** I used AI to help clarify the assignment requirements and to discuss design questions, such as how to handle the mismatch between 10-minute input resolution and hourly prediction resolution.
2. **Code generation:** I used AI to generate code where necessary. I reviewed, tested, and adapted this code myself, and I understand how it works.
3. **Architecture diagram generation:** I used AI to help generate the architecture diagram of the encoder-decoder model, based on the design I specified.

The overall design decisions, implementation, experiments, and analysis are my own work. I take full responsibility for the content of this submission.

Declaration

I confirm that all code submitted for Assignment 2 was written with help from Claude, that all figures/Outputs were produced by running my own completed notebook, and that all prose in the submitted report reflects my own understanding and final wording, informed but not replaced by the AI exchanges logged above.